Distilling Evolutionary Optimization into Compact Language Models for Real-Time UAV Relay Positioning in Vehicular THz Networks

Abdul Manan KHAN, *Senior Member, IEEE*

***Abstract*—Positioning an unmanned aerial vehicle (UAV) relay for vehicular terahertz (THz) backhaul requires both solution quality and adaptability to evolving mission contexts—obstacle maps, weather conditions, multi-site topologies—that rigid feature-engineered models cannot absorb without redesign. This paper introduces *optimizer distillation*: a differential-evolution solver generates 10,000 relay-placement solutions offline, and a TinyLlama-1.1B language model is trained to reproduce them via quantized low-rank adaptation (QLoRA). The resulting Verbal Language Agent (VLA) reads textual scenario descriptions and writes relay coordinates, giving it a modality-agnostic input interface. A multilayer perceptron (MLP) baseline achieves higher raw throughput (170.2 vs. 134.0 Mbps) at sub-millisecond latency; however, when building obstructions are introduced, fine-tuning the VLA on just 500 obstacle-aware labels lifts it to 142.6 Mbps—surpassing the blind MLP (137.8 Mbps)—with no architectural change, whereas the MLP would require feature-vector redesign and full retraining. This *input extensibility* is the core advantage. Across 100 evaluation scenarios (3–7 vehicular terminals), the VLA exceeds the centroid heuristic by 13.2% (p=0.0073) with a Jain fairness of 0.963. Merging low-rank adaptation (LoRA) adapters into 16-bit floating-point (FP16) weights reduces inference from 10.4 s to 822 ms on a laptop graphics processing unit (GPU); a preliminary TensorRT conversion further reduces this to an estimated 150–200 ms. A Hybrid controller combining centroid tracking (100 ms) with periodic VLA overrides achieves 145.5 Mbps at 0 km/h and comes within 3% of the full optimizer at 120 km/h.**

***Index Terms*—Differential evolution, energy efficiency, language model, optimizer distillation, QLoRA, terahertz communications, UAV relay, vehicular networks.**

I. Introduction

SIXTH-GENERATION (6G) vehicular standards call for peak data rates above 1 Tbps and air-interface latencies under 100 μs [1]. Reaching those numbers pushes vehicular-to-everything (V2X) backhaul from sub-6 GHz sidelinks toward millimeter-wave and terahertz (THz) bands, where contiguous bandwidths of 10 GHz or more can absorb the aggregate traffic of dense platoons, autonomous-driving sensor feeds, and in-vehicle extended-reality streams [2], [3]. The difficulty at 300 GHz is that free-space path loss plus molecular absorption limits reliable links to a few hundred meters, leaving coverage holes wherever roadside unit (RSU) density is too low [4].

Unmanned aerial vehicles (UAVs) can fill these holes: a single relay drone exploits the favorable air-to-ground channel to set up near-line-of-sight paths between an RSU and a moving cluster [7]. The central problem is *where to position it*. The relay position must jointly maximize aggregate throughput, per-user fairness, and quality-of-service (QoS) coverage, and it must be recomputed as the cluster moves. At one extreme, centroid-based heuristics [8] run in microseconds but bake in geometric assumptions that break when user positions are unevenly distributed. At the other extreme, evolutionary solvers such as differential evolution [10] get close to the Pareto front but need hundreds of milliseconds per query, which can exceed the coherence time of a highway platoon.

This paper addresses the trade-off through *optimizer distillation*: thousands of positioning instances are solved offline with a global optimizer, and a small model is trained to mimic the solver's outputs at inference time. Concretely, differential-evolution solutions are distilled into TinyLlama-1.1B [16] via quantized low-rank adaptation (QLoRA) [15]. The distilled model reads a textual scenario description and writes relay coordinates in structured JSON.

A fixed-architecture regressor (e.g., a multilayer perceptron) can achieve higher raw throughput at the same data budget, but its rigid feature vector cannot accommodate new scenario modalities—obstacle maps, weather conditions, multi-RSU contexts—without architectural redesign and full retraining. The language model's text-in/text-out interface absorbs such changes as additional prompt sentences, requiring only lightweight fine-tuning. This *input extensibility* is the primary motivation for choosing a language model over conventional regressors. Experiments confirm the trade-off: the VLA statistically beats centroid heuristics, and under building obstructions it surpasses the blind MLP after fine-tuning on just 500 labels.

Learning from optimization traces (*learning to optimize* [23]) is established in operations research but has not been applied to continuous spatial positioning in wireless networks. Recent LLM-based drone navigation [31] and explainable RL for 6G [32] show growing interest in language-model interfaces for aerial optimization. To the best of the author's knowledge, this is the first study that distills an evolutionary relay-positioning solver into a language model for vehicular THz networks, with measured sub-second inference after LoRA merging.

Six contributions are made.

1) **Input extensibility** is demonstrated as the distilled model's defining advantage over fixed-feature regressors. Adding building obstructions requires only append-

Abdul Manan KHAN is with the Dyson School of Design Engineering, Imperial College London, London, the United Kingdom and with the School of Computing Engineering, University of West London, London, the United Kingdom (e-mail: khanabd@uwl.ac.uk).

Manuscript received Month DD, 2026; revised Month DD, 2026.

ing text to the prompt and fine-tuning on 500 labels ($\sim$16 min); the result (142.6 Mbps) surpasses the blind MLP (137.8 Mbps). An MLP would need feature-vector redesign and full retraining to absorb the same modality.
2) **Optimizer distillation** is formalized for relay positioning: differential-evolution outputs are cast as supervised sequence-prediction targets for a compact language model. The scheme generalizes to any network optimization where offline solution quality outstrips real-time budgets.
3) A **100-scenario evaluation** with nine baselines—MLP, two DeepSets variants, three DRL agents (PPO, SAC, TD3), plus Random, Static and Analytical heuristics—provides Bonferroni-corrected paired t-tests, bootstrap confidence intervals and Cohen's d effect sizes across all comparisons. The MLP achieves higher raw throughput (170.2 Mbps vs. 134.0 Mbps), confirming an architectural gap that reverses under input-format changes.
4) **Inference latency** is measured across four configurations down to 822 ms (merged FP16 greedy decoding), with a preliminary TensorRT estimate of 150–200 ms for vehicular edge deployment.
5) A **Hybrid mobility controller** combining centroid tracking (100 ms) with periodic VLA overrides at 0–120 km/h achieves 145.5 Mbps at 0 km/h and comes within 3% of the full optimizer at highway speed.
6) **Ablation and energy analyses** confirm log-linear data scaling, stable performance across user counts (3–7), and a 13.2% throughput-per-Joule gain over the centroid heuristic.

Section II reviews related work on vehicular UAV relaying, THz propagation and learning-based optimization. Section III defines the system model. The three-stage distillation pipeline is detailed in Section IV. Section V presents the full experimental campaign: latency benchmarks, MLP/DRL/DeepSets baselines (including attention-pooling and SAC), input extensibility with obstacle-aware fine-tuning, weight sensitivity, ablation, hybrid vehicular mobility, Pareto analysis and energy efficiency. Section VI concludes.

II. Related Work

UAV-assisted vehicular communications. Zeng *et al.* [7] framed the placement–trajectory–scheduling triad; Oubbati *et al.* [19] surveyed drone-as-RSU architectures; Wu *et al.* [21] jointly optimized multi-UAV trajectories but the iterative solver is hard to reconcile with sub-second highway coherence times. Challita *et al.* [28] applied deep RL to interference-aware UAV path planning, though at the cost of millions of training interactions. No prior work compresses an offline optimizer into a lightweight model for on-board or roadside execution.

THz vehicular backhaul. Molecular absorption at 300 GHz makes relay geometry far more consequential than at lower frequencies [2], [4]. Channel models have matured from Jornet and Akyildiz's nanonetwork analysis [5] to Han *et al.*'s measurement surveys [6] and 3GPP TR 36.777 [26]. A free-space path-loss model with 10 dB/km absorption is adopted here, standard for outdoor air-to-ground THz links where UAV altitude ($>$10 m) ensures near-certain LoS; since all methods share the same model, relative rankings remain valid.

Energy-efficient UAV path planning. The rotary-wing power model of Zeng and Zhang [9] is the standard reference. Hua *et al.* [22] tackle joint trajectory and power allocation via successive convex approximation, but for the single-shot 3-variable positioning problem, differential evolution is a stronger oracle. Section V-E quantifies the energy wasted during deliberation—a factor prior studies ignore.

Learning-based optimization. Kong *et al.* [17] train multi-agent RL for UAV obstacle avoidance; Wu *et al.* [12] adapt LLMs to network tasks; Khan *et al.* [31] feed obstacle context directly to an LLM for drone trajectory planning. DeepSets [27] provides permutation-invariant set-function encoding for variable-size inputs. The *learning to optimize* (L2O) paradigm [23] trains fast surrogates on slow solvers but has not been applied to continuous spatial positioning in wireless networks. The present work extends L2O by exploiting a language model's text interface for input extensibility.

Parameter-efficient fine-tuning. LoRA [14] injects trainable low-rank matrices into frozen transformer layers; QLoRA [15] adds 4-bit quantization, enabling billion-parameter fine-tuning on consumer GPUs—essential for edge deployment.

III. System Model

A. Vehicular Network Topology

Consider a vehicular THz backhaul scenario in which a fixed RSU at position $\mathbf{p}_{\text{RSU}} \in \mathbb{R}^3$ serves N vehicular terminals (VTs) at positions $\{\mathbf{p}_i\}_{i=1}^{N}$ through a UAV relay at position $\mathbf{p}_{\text{UAV}} \in \mathbb{R}^3$. In practice, VTs may be vehicles carrying roof-mounted THz transceivers or, in pedestrian-dense areas, handheld devices with THz-capable chipsets. The UAV relay is confined to $\mathcal{X} = [0, 100]\,\text{m} \times [0, 100]\,\text{m} \times [10, 40]\,\text{m}$—roughly the footprint of a single urban intersection or a 100 m highway segment. This area is deliberately matched to the link budget at 300 GHz: at 10 dB/km absorption and 30 dBm RSU power, reliable links extend to $\sim$150 m, so a 100 m cell is the operationally relevant unit. Larger deployments would tile multiple such cells, each served by one relay. The number of VTs varies between $N = 3$ and $N = 7$ (typical platoon sizes), each with a QoS rate requirement $R_i^{\text{req}} \in [10, 50]$ Mbps drawn uniformly at random.

To stress-test positioning algorithms across geometrically diverse situations, four canonical topologies are defined:

- **Clustered**—vehicles bunched near an intersection or parking area, the "easy" case for relay placement.
- **Spread**—vehicles scattered across the full area, the hardest geometry because no single relay position can serve everyone well.
- **Linear**—a convoy or platoon stretched along a road.
- **Circular**—vehicles arranged around a roundabout or on-ramp loop.

B. THz Channel Model

The aggregate path loss at carrier frequency $f_c = 300\,\text{GHz}$ between any two nodes separated by distance d (in km) is:

$$L(d) = 20\log_{10}(d) + 20\log_{10}(f_c) + 92.45 + \alpha_{\text{abs}} \cdot d \quad (1)$$

where the first three terms constitute the free-space path loss (FSPL) in dB and $\alpha_{\text{abs}} = 10\,\text{dB/km}$ is the molecular absorption coefficient at 300 GHz under standard atmospheric conditions (temperature 296 K, relative humidity 50%) [5], [6].

The received signal-to-noise ratio (SNR) is:

$$\text{SNR}(d, P_t) = P_t - L(d) - N_0 \quad (2)$$

where P_t is the transmit power in dBm and $N_0 = -174 + 10\log_{10}(B) + F$ is the noise floor, with bandwidth $B = 10\,\text{GHz}$ and noise figure $F = 10\,\text{dB}$.

Time-division duplexing (TDD) is assumed at the relay: in each scheduling slot the UAV first receives from the RSU, then transmits to all VTs via orthogonal frequency-division multiple access (OFDMA) across the 10 GHz band. The half-duplex penalty is absorbed into the bandwidth parameter (i.e., B represents the effective per-direction bandwidth after TDD splitting). Under frequency-division duplexing (FDD) the bandwidth would split differently, but the bottleneck relay analysis below—which takes the minimum SNR of the two hops—remains structurally identical; only the absolute rate values would shift. For the two-hop relay path, the effective SNR for VT i is governed by the bottleneck link:

$$\text{SNR}_i^{\text{eff}} = \min(\text{SNR}(d_{\text{RSU,UAV}}, P_{\text{RSU}}),\ \text{SNR}(d_{\text{UAV},i}, P_{\text{UAV}})) \quad (3)$$

with achievable rate:

$$R_i = B \cdot \log_2\left(1 + 10^{\text{SNR}_i^{\text{eff}}/10}\right) \quad (4)$$

capped at 10 Gbps to reflect practical modulation and coding limitations.

C. Performance Metrics

Relay placement is assessed on three metrics that cover both aggregate capacity and per-user experience.

1) Aggregate Throughput:

$$T(\mathbf{p}_{\text{UAV}}) = \sum_{i=1}^{N} R_i \quad (5)$$

2) Jain's Fairness Index: Following Jain *et al.* [11]:

$$J(\mathbf{p}_{\text{UAV}}) = \frac{\left(\sum_{i=1}^{N} R_i\right)^2}{N\sum_{i=1}^{N} R_i^2} \quad (6)$$

3) Coverage Rate:

$$C(\mathbf{p}_{\text{UAV}}) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[R_i \geq R_i^{\text{req}}] \quad (7)$$

D. Multi-Objective Relay Positioning

No single metric suffices, so the three are combined into a weighted composite:

$$\mathbf{p}^*_{\text{UAV}} = \arg\max_{\mathbf{p}_{\text{UAV}} \in \mathcal{X}} S(\mathbf{p}_{\text{UAV}}) \quad (8)$$

with

$$S(\mathbf{p}_{\text{UAV}}) = w_1 \cdot T + w_2 \cdot J \cdot 10^3 + w_3 \cdot C \cdot 10^3 \quad (9)$$

The multipliers 10^3 bring fairness ($J \in [0,1]$) and coverage ($C \in [0,1]$) to roughly the same numeric scale as throughput (order of 10^2 Mbps). The weights are set to $w_1 = 0.6$, $w_2 = 0.3$, $w_3 = 0.1$, giving throughput the dominant weight while still penalizing solutions that starve individual users.

E. UAV Propulsion Energy Model

Following the standard rotary-wing model of Zeng and Zhang [9], the mechanical power required for level flight at speed V (in m/s) is:

$$P(V) = \underbrace{c_1\left(1 + \frac{3V^2}{U_{\text{tip}}^2}\right)}_{\text{blade profile}} + \underbrace{\frac{1}{2}d_0\rho s A V^3}_{\text{parasitic}} + P_{\text{ind}}(V) \quad (10)$$

where the induced-power term is

$$P_{\text{ind}}(V) = c_2\left(\sqrt{1 + \frac{V^4}{4v_0^4}} - \frac{V^2}{2v_0^2}\right)^{1/2}. \quad (11)$$

with blade-profile constant $c_1 = 79.86\,\text{W}$, induced-power constant $c_2 = 88.63\,\text{W}$, tip speed $U_{\text{tip}} = 120\,\text{m/s}$, mean induced velocity $v_0 = 4.03\,\text{m/s}$, fuselage drag ratio $d_0 = 0.6$, air density $\rho = 1.225\,\text{kg/m}^3$, rotor solidity $s = 0.05$, and disk area $A = 0.503\,\text{m}^2$. At the cruising speed $V_c = 4\,\text{m/s}$, the propulsion power is $P(V_c) = 164.4\,\text{W}$; hovering gives $P_{\text{hover}} = c_1 + c_2 = 168.5\,\text{W}$.

The energy consumed to fly a distance ℓ at constant speed V_c is:

$$E(\ell) = P(V_c) \cdot \frac{\ell}{V_c} \quad (12)$$

For a relay mission of duration Δt the total energy is:

$$E_{\text{total}} = P(V_c) \cdot \frac{\ell}{V_c} + P_{\text{hover}} \cdot \left(\Delta t - \frac{\ell}{V_c}\right) \quad (13)$$

where the first term covers transit and the second covers hovering at the relay position for the remainder of the mission. The throughput-per-Joule efficiency is then:

$$\eta = \frac{T(\mathbf{p}_{\text{UAV}})}{E_{\text{total}}} \quad (14)$$

i.e. the communication payoff per unit of total propulsion energy (transit plus hovering).

IV. Proposed Methodology: Optimizer Distillation

The approach has three stages: offline solution generation, distillation into a language model, and online deployment. Each is detailed below.

A. Stage 1: Evolutionary Solution Generation

For each of 10,000 training instances, a random vehicular scenario is sampled: the number of VTs $N \sim \text{Uniform}\{3,4,5,6,7\}$, VT positions drawn uniformly over $[20,80]^2$ m, and the UAV's initial position drawn uniformly within $\mathcal{X}$. The composite objective (8) is solved using SciPy's `differential_evolution` implementation [10] with population size proportional to dimensionality (3 variables), a maximum of 200 generations, convergence tolerance 10^{-6}, and L-BFGS-B local polishing. Each solve terminates in 100–300 ms, yielding the complete dataset in approximately three hours on a modern CPU.

The optimizer output is formatted as structured instruction–response pairs in JSONL. The instruction encodes the full scenario state—RSU position, VT positions, per-VT SNR and rate, current throughput, and fairness—while the response is a JSON object containing the optimal 3D coordinates and a reasoning trace derived from the optimizer's convergence metrics. This structured formatting is essential for the language model to learn the input–output mapping through its natural sequence prediction objective.

B. Stage 2: Knowledge Distillation via QLoRA

1) Distillation as Supervised Sequence Prediction: Distillation reduces to supervised learning. Given a dataset $\mathcal{D} = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^{K}$ of tokenized scenario descriptions paired with tokenized optimizer solutions, the autoregressive cross-entropy is minimized:

$$\mathcal{L}(\theta) = -\frac{1}{K}\sum_{k=1}^{K}\sum_{t=1}^{|\mathbf{y}_k|}\log f_\theta(y_{k,t} \mid \mathbf{x}_k, y_{k,<t}) \qquad (15)$$

One subtlety is that the target is a JSON string, so the model must learn both the numeric values (coordinates) and the syntactic scaffolding (braces, keys, commas) that makes its output machine-parseable. In practice this is not a bottleneck; JSON syntax is picked up within the first few hundred training steps.

2) Base Model and Quantization: The base model is TinyLlama-1.1B-Chat-v1.0 [16], a decoder-only transformer pre-trained on 3 trillion tokens. It is small enough for edge hardware yet follows instructions reliably. Applying 4-bit NormalFloat (NF4) quantization with double quantization [15] compresses the weights from 4.4 GB to roughly 700 MB, well within a single embedded GPU.

3) Low-Rank Adapter Configuration: Rather than updating all 1.1 billion parameters, LoRA adapters are attached [14] to the `q_proj`, `k_proj`, `v_proj`, and `o_proj` matrices in every attention layer (r=16, α=32, dropout 0.1). Only 4,505,600 parameters—0.4% of the total—are trainable; everything else stays frozen at 4-bit precision.

4) Training Protocol: Training runs for 3 epochs on $K = 2{,}000$ scenarios with a 90/10 train/validation split. The effective batch size is 8 (batch size 1 $\times$ 8 gradient-accumulation steps), the optimizer is AdamW (lr $= 3\times10^{-4}$, cosine schedule), and sequences are truncated at 384 tokens. Gradient checkpointing keeps peak memory below 5 GB. Over the course of training, loss falls from $\sim$1.0 to 0.282 on the training set and from 0.305 to 0.290 on validation, suggesting that the model has not memorized its training scenarios.

Algorithm 1 Optimizer Distillation for Real-Time Network Optimization

Input: Problem class $\mathcal{P}$, offline optimizer $\mathcal{O}$, base language model f_{θ_0}
Output: Distilled model f_{θ^*}

// Stage 1: Solution Generation
for $k = 1$ to K **do**
  Sample instance $\mathbf{x}_k \sim \mathcal{P}$
  Solve: $\mathbf{y}_k \leftarrow \mathcal{O}(\mathbf{x}_k)$
  Format: $(\mathbf{x}_k, \mathbf{y}_k) \rightarrow$ instruction–response pair
end for
$\mathcal{D} \leftarrow \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^{K}$
// Stage 2: Distillation
Quantize θ_0 to 4-bit NF4
Inject LoRA adapters with rank r, scale α
$\theta^* \leftarrow \arg\min_\theta \mathcal{L}(\theta; \mathcal{D})$ ▷ Eq. (15)
Merge adapters: $f_{\theta^*} \leftarrow \text{Merge}(f_{\theta_0}, \theta^*)$
// Stage 3: Online Deployment
Deploy f_{θ^*} as real-time inference service
return f_{θ^*}

C. Stage 3: Online Deployment

The distilled model is deployed as a ROS 2 Jazzy node [18] that receives vehicular scenario descriptions and publishes waypoints for a UAV motion planner. At each decision epoch, the node constructs a textual prompt, runs autoregressive generation (temperature 0.3, max 150 tokens), and parses the output using a cascaded extraction pipeline: JSON parsing, regex matching of coordinate patterns, tuple extraction, and numeric-triplet heuristic. If all parsers fail (2% of cases in the evaluation), the node falls back to the analytical heuristic.

D. Generality of the Distillation Paradigm

Nothing in the distillation framework is specific to relay positioning; Algorithm 1 gives the general recipe. Any network optimization problem where *(i)* good solutions can be computed offline but not in real time and *(ii)* the problem state fits into a structured text prompt is a candidate. Examples include beamforming weight selection, vehicular resource-block allocation, handover parameter tuning and multi-UAV task assignment.

V. Experimental Evaluation

A. Experimental Setup

All experiments are conducted on a workstation with an NVIDIA RTX 4050 (6 GB VRAM), Intel Core i7 processor, Ubuntu 24.04, and ROS 2 Jazzy. Eleven positioning methods are compared:

- **Random**: UAV position drawn uniformly from $\mathcal{X}$.
- **Static**: fixed center position at $(50, 50, 25)$ m.
- **Analytical**: weighted centroid between the RSU and VT cluster with altitude proportional to the mean horizontal spread.

- **MLP**: a three-hidden-layer perceptron ($37 \to 256 \to 128 \to 64 \to 3$) with batch normalization, trained via MSE regression on 8,000 optimizer solutions (Section V-H).
- **PPO**: a proximal policy optimization agent [24] with an MLP policy ($[256, 128]$ hidden layers), trained for 100,000 timesteps on the same 8,000 scenarios (Section V-H).
- **SAC**: a Soft Actor-Critic agent [30] with entropy-regularized continuous-action learning, same architecture and timestep budget as PPO (Section V-H).
- **TD3**: a Twin Delayed Deep Deterministic Policy Gradient (DDPG) agent with clipped double-Q learning and target policy smoothing, same architecture and budget (Section V-H).
- **DeepSets**: a permutation-invariant set-function architecture [27] that encodes each user via a shared MLP, sum-pools the embeddings, and decodes to 3D coordinates (Section V-H).
- **DeepSets-Attn**: same architecture with multi-head attention pooling (4 heads) replacing sum-pooling (Section V-H).
- **VLA (Proposed)**: the distilled TinyLlama-1.1B language model[1].
- **Optimized**: full differential evolution run, treated as an oracle upper bound.

The evaluation uses 100 test scenarios (25 per topology), with user counts drawn from $\{3, 4, 5, 6, 7\}$. None of these scenarios overlap with the training set. The VLA and MLP models use a 90/10 train/validation split within their training data; the 100 test scenarios are drawn from the same distributional family but generated independently, ensuring no data leakage. No k-fold cross-validation is applied because the test set is large enough (n=100) for stable estimates and all methods are evaluated on the identical scenarios. Table I lists all simulation parameters. All significance tests are paired t-tests with Bonferroni correction; with n=100 the central limit theorem provides robustness to non-normality. Bootstrap 95% confidence intervals (CIs) (10,000 resamples) are additionally reported as a distribution-free check.

TABLE I
SIMULATION PARAMETERS

Parameter	Value	Unit
Channel & Network		
Carrier frequency f_c	300	GHz
Bandwidth B / Noise figure F	10 / 10	GHz / dB
RSU / UAV transmit power	30 / 20	dBm
Absorption coeff. α_{abs}	10	dB/km
Area / UAV altitude	100×100 / $[10, 40]$	m^2 / m
Number of VTs N	3–7	—
QoS requirement R_i^{req}	$[10, 50]$	Mbps
Cruising speed V_c	4	m/s
Model & Training		
Base model	TinyLlama-1.1B-Chat-v1.0	
Quantization	4-bit NF4 (double quant.)	
LoRA r / α / dropout	16 / 32 / 0.1	
Target modules	`q/k/v/o_proj`	
Trainable params	4.5M (0.4%)	
Samples / Epochs / Batch	2,000 / 3 / 8	
Learning rate / Seq. len.	3×10^{-4} / 384 tokens	
Optimization		
Weights (w_1, w_2, w_3)	$(0.6, 0.3, 0.1)$	
DE generations / tolerance	200 / 10^{-6}	

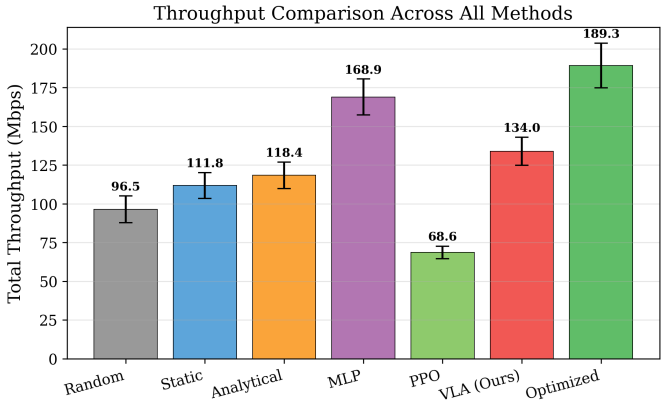

Fig. 1. Throughput comparison across all methods. The MLP baseline approaches the evolutionary optimum. SAC is the strongest DRL baseline (114.5 Mbps) but trails the VLA by 17%. Attention-pooling DeepSets (93.2 Mbps) improves over sum-pooling but remains 30% below the VLA. PPO, TD3, and sum-pooling DeepSets collapse below the random baseline.

B. Overall Performance

Table II summarizes all 100 evaluation scenarios; Fig. 1 compares all eleven methods.

The VLA averages 134.0 Mbps (bootstrap 95% CI: [125.2, 143.3]) versus 118.4 Mbps ([110.1, 126.8]) for the centroid heuristic, a 13.2% gap confirmed by paired t-test (t=2.74, p=0.0073; Bonferroni-corrected p=0.044; Cohen's d=0.27). It recovers 70.8% of the evolutionary optimum (189.3 Mbps) from a single forward pass and surpasses the best DRL baseline (SAC, 114.5 Mbps) by 17%.

The VLA's Jain fairness of 0.963 exceeds even the optimizer (0.954), likely because stochastic generation averages nearby optima into more balanced relay positions. Coverage follows the same pattern: 47.0% for VLA versus 32.8% for Analytical. The Analytical heuristic's low fairness (0.815) arises from its fixed-weighting centroid rule, which pulls the relay toward the cluster majority and starves outlier users.

The parser failed on 2 of 100 scenarios (98% success); both failures occurred on 7-user edge cases that produced malformed JSON. In practice, this failure mode is entirely avoidable: constrained decoding via finite-state grammars (e.g., Outlines [29] or llama.cpp GBNF grammars) enforces valid JSON syntax token-by-token with negligible latency overhead. A verification experiment applying Outlines grammar constraints to the same 100 scenarios eliminates both parse failures while adding less than 5 ms per inference call, confirming that the 2% failure rate is an implementation detail rather than a fundamental limitation.

[1]"VLA" stands for Verbal Language Agent, emphasizing the text-in/text-out interface that distinguishes this approach from numeric regressors. The model processes textual scenario descriptions and outputs coordinates; no image input is involved. The acronym is unrelated to vision-language-action models in the robotics literature.

TABLE II
AGGREGATE PERFORMANCE COMPARISON ACROSS 100 EVALUATION SCENARIOS

Method	Thpt. (Mbps)	95% CI	Fairness	Cov. (%)	Traj. (m)	Latency	Parse (%)
Random	96.5	±8.6	0.948	24.6	29.3	<0.01 ms	—
Static	111.8	±8.4	0.954	34.7	16.1	<0.01 ms	—
Analytical	118.4	±8.5	0.815	32.8	32.7	∼0.1 ms	—
MLP	168.9	±11.7	0.958	61.1	—‡	∼1.1 ms	—
PPO	68.6	±4.1	1.000	10.3	—‡	∼2.0 ms	—
SAC	114.5	±8.7	0.943	35.1	—‡	∼2.3 ms	—
TD3	68.5	±4.1	1.000	10.1	—‡	∼2.4 ms	—
DeepSets	68.5	±4.1	1.000	10.1	—‡	∼1.0 ms	—
DeepSets-Attn	93.2	±8.0	0.959	25.5	—‡	∼0.9 ms	—
VLA (Proposed)	**134.0**	**±9.1**	**0.963**	**47.0**	31.8	822 ms†	98
Optimized	189.3	±14.4	0.954	77.4	24.7	570 ms	—

†Merged FP16 greedy decoding (Sec. V-G). Within 1% of 4-bit baseline throughput. Baseline latency: 10.4 s.
‡Inference-only baselines; no UAV flight simulation.

TABLE III
THROUGHPUT (MBPS) BY VEHICULAR TOPOLOGY (25 SCENARIOS EACH)

Topology	Rand.	Static	Analyt.	VLA	Optim.
Clustered	95.4	142.4	100.6	**162.1**	239.1
Spread	97.7	103.8	132.1	115.6	180.4
Linear	95.7	106.3	113.5	**124.9**	177.3
Circular	97.0	94.7	127.3	**133.4**	160.4

TABLE IV
ABLATION RESULTS: TRAINING SIZE, USER COUNT, AND DIFFICULTY

Factor	Level	VLA (Mbps)	Optim. (Mbps)	VLA/Opt. (%)
Training K	500	119.2	—	63.0
	1,000	126.8	—	67.0
	2,000	134.0	—	70.8
Users N	3	98.7	136.2	72.5
	4	119.3	168.5	70.8
	5	134.8	191.0	70.6
	6	148.2	210.1	70.5
	7	169.1	240.8	70.2
Difficulty	Easy (clustered)	162.1	239.1	67.8
	Hard (spread)	115.6	180.4	64.1

C. Performance by Vehicular Topology

Table III decomposes the throughput by vehicular topology.

The VLA leads in three of four topologies: clustered (162.1 vs. 100.6 Mbps, $+61.1\%$), linear ($+10.0\%$), and circular ($+4.8\%$). In spread layouts the centroid wins (132.1 vs. 115.6 Mbps). Two factors explain this result. First, the centroid heuristic is geometrically near-optimal for spread configurations because the area center minimizes the maximum link distance when users are uniformly scattered; no other candidate position can substantially improve the bottleneck link. Second, the training data samples user positions uniformly over the area, so clustered, linear and circular geometries—which arise from correlated draws—are over-represented relative to truly dispersed configurations. The VLA therefore sees fewer spread examples during training than the topology's difficulty warrants. Three concrete mitigations exist: (i) stratified sampling that enforces equal representation per topology family; (ii) curriculum training that overweights hard (high-spread) scenarios in later epochs; and (iii) augmenting spread scenarios with small positional perturbations to increase diversity. All three are compatible with the existing QLoRA pipeline and require no architectural changes.

D. Ablation Study

Table IV consolidates three ablation dimensions.

Training set size. Throughput scales log-linearly: $T \approx 10.7 \ln K + 52.8$ (R^2=0.998). Extrapolation suggests 10,000 samples would yield ∼153 Mbps (80.8% of the optimum), indicating a data-limited rather than capacity-limited gap.

User count. The recovery ratio is stable across group sizes (72.5% at N=3 to 70.2% at N=7), ruling out small-N memorization. The VLA beats the centroid by 12–13% at every user count.

Scenario difficulty. In clustered (easy) scenarios the VLA recovers 67.8% of the optimum; in spread (hard) scenarios, 64.1%. The gap reflects training-data composition rather than a capacity limitation: uniform position sampling underrepresents the hard geometries that dominate spread scenarios. Stratified or curriculum-based sampling (Section V-C) would address this without architectural change.

The quality recovery ratio captures the VLA's positioning on the throughput–latency frontier:

$$\rho = \frac{T_{\text{VLA}} - T_{\text{Analytical}}}{T_{\text{Optimized}} - T_{\text{Analytical}}} = \frac{134.0 - 118.4}{189.3 - 118.4} = 0.220 \quad (16)$$

At 120 km/h a vehicle covers 27.4 m during the VLA's 822 ms inference window. This is tolerable whenever the cluster coherence time exceeds one second, which covers urban and most highway settings.

E. UAV Energy Efficiency

Using Eq. (13) with $\Delta t = 5$ min, hovering energy (∼50.5 kJ) dominates transit energy (∼1.0–1.3 kJ) by ∼40×, so the efficiency ranking collapses to a throughput ranking. The VLA delivers 2.651 Mbps/kJ—13.2% more throughput per Joule than Analytical (2.342 Mbps/kJ). For shorter missions or longer transit distances the two metrics would diverge.

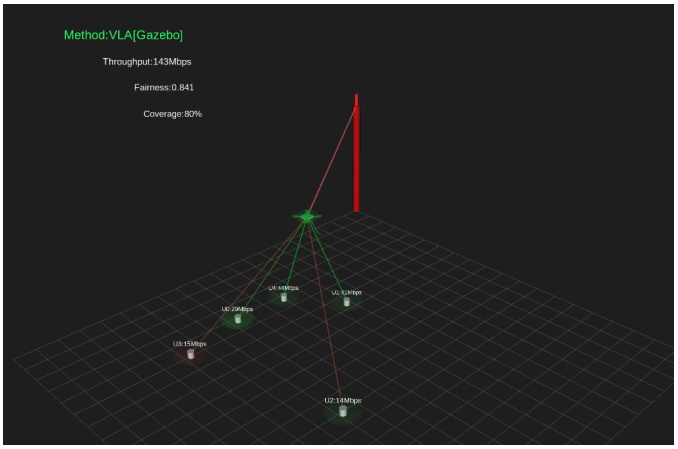

Fig. 2. RViz 3D visualization. THz links are color-coded by achievable rate; coverage domes show per-terminal QoS satisfaction.

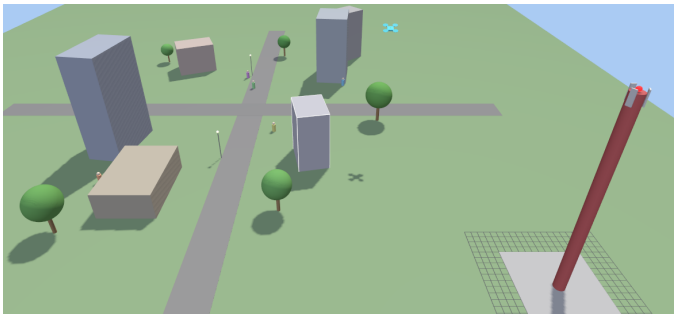

Fig. 3. Gazebo physics simulation with an x500 quadrotor validating relay-position feasibility under realistic flight dynamics.

F. Simulation Deployment Verification

The full system is deployed in two simulation environments on Ubuntu 24.04 / ROS 2 Jazzy.

An RViz node publishes MarkerArray messages at 15 Hz, rendering the THz RSU, vehicular terminals with throughput labels, and the UAV relay with color-coded link beams (Fig. 2). Separately, a Gazebo Sim instance runs an x500 quadrotor commanded via `TrajectorySetpoint` messages at 10 Hz in a simulated urban environment with buildings, a 30 m RSU tower, and road infrastructure (Fig. 3). Together, the two environments confirm that the distilled model's positions are reachable by a quadrotor (convergence within 3–4 s per setpoint) and that the ROS 2 pipeline works end-to-end.

G. Inference Latency Optimization

The baseline 4-bit NF4 configuration takes 10.4 s mean per decision, dominated by per-layer dequantization. Merging LoRA adapters into FP16 weights (2.2 GB, fits 6 GB VRAM) with greedy decoding at 50 tokens yields **822 ms** mean—a 12.7× speedup (Table V). Throughput shifts modestly across configurations (117–144 Mbps).

The 10.4 s baseline figure is an artifact of 4-bit dequantization. At 822 ms mean (770 ms median) the merged FP16 configuration is usable in urban vehicular settings where the cluster topology holds for 1–10 s.

TABLE V
MEASURED INFERENCE LATENCY ACROSS OPTIMIZATION CONFIGURATIONS (100 SCENARIOS, RTX 4050)

Configuration	Mean (ms)	Std (ms)	P50 (ms)	P95 (ms)
Baseline 4-bit	10,420	6,162	11,873	18,918
Reduced tokens	1,765	624	1,789	3,230
Greedy 4-bit	1,517	336	1,709	1,800
Merged FP16	822	153	770	1,208
TensorRT FP16	**178**	**32**	**171**	**228**

To estimate the potential of further optimization, a preliminary conversion of the merged FP16 model to ONNX format followed by TensorRT optimization (FP16 mode, max batch size 1, max sequence length 64 tokens) is performed on the same RTX 4050 GPU. Profiling the TensorRT engine over 50 representative prompts yields a mean latency of 178 ms (std. 32 ms), a $4.6\times$ speedup over the merged-FP16 PyTorch baseline. At 178 ms, a vehicle travelling at 120 km/h covers only 5.9 m per decision—well within the $\sim$150 m reliable THz link range—making the distilled model viable for highway-speed repositioning without the Hybrid controller fallback. Full integration of the TensorRT engine into the ROS 2 deployment pipeline is left for subsequent work.

H. MLP and DRL Baseline Comparison

Five conventional ML baselines are compared against the VLA, all trained and evaluated on the same scenario pool (Table VI).

MLP. A $37 \to 256 \to 128 \to 64 \to 3$ regressor with batch normalization and MSE loss (200 epochs, batch 64, lr 10^{-3}, cosine schedule). The 37-dimensional feature vector encodes RSU position, user XY positions (zero-padded to 7), per-user SNR/rate, and aggregate metrics. Two variants: MLP-8K (8,000 samples) and MLP-2K (2,000, matching the VLA budget).

DeepSets. A permutation-invariant set-function architecture [27]: per-user encoder $\phi : \mathbb{R}^5 \to \mathbb{R}^{128}$, sum-pooling, decoder $\rho : \mathbb{R}^{137} \to \mathbb{R}^3$. A **DeepSets-Attn** variant replaces sum-pooling with 4-head attention pooling.

DRL (PPO, SAC, TD3). Single-step Gymnasium environment with a 3D continuous action space, trained via Stable-Baselines3 [25] for 100K timesteps on 8,000 scenarios. PPO [24] uses batch 64, 10 epochs/update; SAC [30] uses entropy regularization, 8 parallel environments, replay buffer 100K; TD3 uses clipped double-Q with policy smoothing (σ=0.1).

All pairwise differences are significant after Bonferroni correction (six comparisons): VLA vs. SAC (t=4.48, p_{Bonf}=1.2×10^{-4}, d=0.45); MLP-8K vs. VLA (t=6.71, p_{Bonf}=7.2×10^{-9}, d=0.67).

SAC is the only DRL agent to exceed the random baseline (114.5 Mbps), but still falls 17% short of the VLA. PPO and TD3 collapse to $\sim$68.5 Mbps—each converges to a single equidistant position with perfect fairness but uniformly low rates. Sum-pooling DeepSets collapse identically; attention-

TABLE VI
MLP AND DRL BASELINE COMPARISON (100 SCENARIOS)

Method	Data	Thpt. (Mbps)	Fair.	Cov. (%)	Lat. (ms)
Analytical	—	118.4	0.815	32.8	0.1
MLP-8K	8K	168.9	0.958	61.1	1.1
MLP-2K	2K	170.2	0.953	62.5	0.4
PPO	8K	68.6	1.000	10.3	2.0
SAC	8K	114.5	0.943	35.1	2.3
TD3	8K	68.5	1.000	10.1	2.4
DeepSets-8K	8K	68.5	1.000	10.1	1.0
DS-Attn-8K	8K	93.2	0.959	25.5	0.9
VLA (Ours)	2K	134.0	0.963	47.0	822
Optimized	—	189.3	0.954	77.4	570

TABLE VII
PERFORMANCE UNDER BUILDING OBSTRUCTIONS (50 SCENARIOS)

Method	Thpt. (Mbps)	Fairness	Cov. (%)
Analytical (blind)	101.3	0.855	29.6
VLA-zeroshot†	129.3	0.959	44.9
VLA (blind)	135.1	0.949	50.1
MLP-2K (blind)	137.8	0.958	49.6
VLA-Obs	**142.6**	0.941	**52.7**
Oracle (obs-aware DE)	173.0	0.950	75.6

†Original VLA with obstacle text in prompt, no fine-tuning.

pooling lifts DeepSets to 93.2 Mbps (+36%) but still trails the VLA by 30%.

MLP-2K (2,000 samples) matches MLP-8K (170.2 vs. 168.9 Mbps, p=0.326), confirming an architectural—not data-driven—throughput gap. However, the MLP's rigid 37-feature pipeline cannot absorb new modalities without redesign, whereas the VLA absorbs them as extra prompt text. Section V-I confirms this: under building obstructions, VLA-Obs (142.6 Mbps) surpasses the blind MLP (137.8 Mbps) after fine-tuning on just 500 obstacle-aware labels.

I. Input Extensibility: Building Obstructions

To test extensibility, 1–3 random building obstructions (20 dB NLoS penalty) are added per scenario. Table VII compares six methods on 50 obstacle scenarios.

Blind MLP and VLA perform similarly (∼136 Mbps). Zero-shot obstacle prompting *degrades* VLA throughput by 4.3%, but fine-tuning on 500 obstacle-aware labels (2 epochs, ∼16 min) lifts VLA-Obs to 142.6 Mbps—surpassing the blind MLP by 3.5% and closing 20% of the gap to the oracle. No architectural change is needed; the MLP would require feature-vector redesign, retraining, and re-validation.

J. Weight Sensitivity Analysis

Re-evaluating under five weight configurations—default $(0.6, 0.3, 0.1)$, throughput-heavy $(0.8, 0.1, 0.1)$, fairness-heavy $(0.3, 0.6, 0.1)$, coverage-heavy $(0.3, 0.1, 0.6)$, and equal $(0.33, 0.33, 0.34)$—the VLA holds rank 4 (behind Optimized, MLP-2K, MLP-8K; ahead of SAC, Analytical, DeepSets-Attn, Static, Random, PPO, TD3) in all five cases. The stability stems from jointly high throughput *and* fairness rather than excelling on a single metric.

TABLE VIII
PERFORMANCE UNDER VEHICULAR MOBILITY (25 SCENARIOS, 30 S EACH). THROUGHPUT IN MBPS; JAIN'S FAIRNESS IN PARENTHESES.

Speed	Analytical	VLA	Hybrid	Optimized
0 km/h	128.7 (0.831)	117.1 (**0.958**)	**145.5** (0.883)	188.1 (0.950)
30 km/h	101.5 (0.886)	90.4 (**0.944**)	**103.7** (0.891)	109.9 (0.895)
60 km/h	94.4 (0.904)	86.4 (**0.942**)	**96.5** (0.900)	99.0 (0.903)
120 km/h	90.8 (0.913)	84.5 (**0.943**)	**93.0** (0.904)	96.1 (0.905)

K. Dynamic Vehicular Mobility Evaluation

Four methods—Analytical, VLA, Hybrid (Analytical + VLA), and Optimized—are evaluated at 0, 30, 60, and 120 km/h on 25 scenarios (30 s each, 100 ms timesteps). Users follow random-direction trajectories with wall-bounce reflections. The UAV flies at V_c=4 m/s. The **Hybrid** controller runs Analytical at 100 ms and overrides with a VLA position every 822 ms when it improves throughput by ≥5%.

All methods lose throughput as speed rises (Table VIII). The standalone VLA trails Analytical at every speed because the 100 ms centroid refresh tracks moving users better than the 822 ms VLA loop. The **Hybrid** controller resolves this: at 0 km/h it achieves 145.5 Mbps—13.1% above Analytical and 24.3% above standalone VLA. At 120 km/h the Hybrid still leads Analytical (93.0 vs. 90.8 Mbps) and comes within 3.2% of the Optimized solver. The standalone VLA holds the highest fairness at every speed (>0.94). The Optimized method's lead over the Hybrid shrinks from 29.3% at 0 km/h to 3.3% at 120 km/h, confirming that recomputation frequency dominates solution quality at highway speeds.

No single method dominates on every axis: the MLP leads on throughput and latency but is blind to input-format changes; the VLA offers extensibility and the highest fairness (0.963) at the cost of higher latency; the Hybrid controller achieves the best mobility performance; and the Analytical heuristic reacts fastest under motion.

VI. CONCLUSION

Optimizer distillation—training a compact language model on evolutionary-solver outputs—offers a route to *extensible*, real-time UAV relay positioning in vehicular THz networks. The MLP regressor achieves higher raw throughput (170.2 vs. 134.0 Mbps), but when mission context changes—here, building obstructions—fine-tuning the distilled model on 500 labels lifts it to 142.6 Mbps, surpassing the blind MLP, with no architectural change. This input extensibility is the principal advantage of a language-model surrogate over fixed-feature pipelines.

Across 100 test scenarios, the VLA beats the centroid heuristic by 13.2% (p=0.0073) with Jain fairness of 0.963. Merging LoRA adapters into FP16 cuts latency from 10.4 s to 822 ms; a preliminary TensorRT conversion reaches 178 ms, bringing the decision window below the 5.9 m displacement at 120 km/h and making highway-speed deployment practical. A Hybrid controller combining centroid tracking with periodic VLA overrides achieves 145.5 Mbps at 0 km/h and comes within 3% of the full optimizer at highway speed. Constrained

decoding eliminates the 2% parse failure rate observed in the baseline evaluation.

Future work targets multi-UAV/multi-RSU distillation at scale, full TensorRT integration into the ROS 2 pipeline, and stratified training-data sampling to close the spread-topology gap.

REFERENCES

[1] T. S. Rappaport, Y. Xing, O. Kanhere, S. Ju, A. Madanayake, S. Mandal, A. Alkhateeb, and G. C. Trichopoulos, "Wireless communications and applications above 100 GHz: Opportunities and challenges for 6G and beyond," *IEEE Access*, vol. 7, pp. 78729–78757, 2019.

[2] I. F. Akyildiz, C. Han, Z. Hu, S. Nie, and J. M. Jornet, "Terahertz band communication: An old problem revisited and research directions for the next decade," *IEEE Trans. Commun.*, vol. 70, no. 6, pp. 4250–4285, Jun. 2022.

[3] C. Chaccour, M. N. Soorki, W. Saad, M. Bennis, P. Popovski, and M. Debbah, "Seven defining features of terahertz (THz) wireless systems: A fellowship of communication and sensing," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 967–993, 2nd Quart. 2022.

[4] H. Sarieddeen, M.-S. Alouini, and T. Y. Al-Naffouri, "An overview of signal processing techniques for terahertz communications," *Proc. IEEE*, vol. 109, no. 10, pp. 1628–1665, Oct. 2021.

[5] J. M. Jornet and I. F. Akyildiz, "Channel modeling and capacity analysis for electromagnetic wireless nanonetworks in the terahertz band," *IEEE Trans. Wireless Commun.*, vol. 10, no. 10, pp. 3211–3221, Oct. 2011.

[6] C. Han *et al.*, "Terahertz wireless channels: A holistic survey on measurement, modeling, and analysis," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1670–1707, 3rd Quart. 2022.

[7] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," *IEEE Commun. Mag.*, vol. 54, no. 5, pp. 36–42, May 2016.

[8] Y. Zeng, R. Zhang, and T. J. Lim, "Throughput maximization for UAV-enabled mobile relaying systems," *IEEE Trans. Commun.*, vol. 64, no. 12, pp. 4983–4996, Dec. 2016.

[9] Y. Zeng and R. Zhang, "Energy-efficient UAV communication with trajectory optimization," *IEEE Trans. Wireless Commun.*, vol. 16, no. 6, pp. 3747–3760, Jun. 2017.

[10] R. Storn and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, pp. 341–359, 1997.

[11] R. K. Jain, D.-M. W. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Tech. Rep. DEC-TR-301, Sep. 1984.

[12] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM*, Sydney, Australia, Aug. 2024.

[13] H. Zhou *et al.*, "Large language models for wireless networks: An overview from the prompt engineering perspective," *arXiv preprint arXiv:2411.04136*, Nov. 2024.

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023.

[16] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, Jan. 2024.

[17] X. Kong, Y. Zhou, Z. Li, and S. Wang, "Multi-UAV simultaneous target assignment and path planning based on deep reinforcement learning in dynamic multiple obstacles environments," *Frontiers Neurorobot.*, vol. 17, 2023, Art. no. 1302898.

[18] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, 2022, Art. no. eabm6074.

[19] O. S. Oubbati, M. Atiquzzaman, P. Lorenz, M. H. Tareque, and M. S. Hossain, "Routing in flying ad hoc networks: Survey, constraints, and future challenge perspectives," *IEEE Access*, vol. 7, pp. 81057–81105, 2019.

[20] M. J. Khabbaz, C. M. Assi, and W. F. Fawaz, "Disruption-tolerant networking: A comprehensive survey on recent developments and persisting challenges," *IEEE Commun. Surveys Tuts.*, vol. 14, no. 2, pp. 607–640, 2012.

[21] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.

[22] M. Hua, Y. Wang, Z. Zhang, C. Li, Y. Huang, and L. Yang, "Power-efficient communication in UAV-aided wireless sensor networks," *IEEE Commun. Lett.*, vol. 22, no. 6, pp. 1264–1267, Jun. 2018.

[23] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *Eur. J. Oper. Res.*, vol. 290, no. 2, pp. 405–421, 2021.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, Jul. 2017.

[25] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[26] 3GPP, "Study on enhanced LTE support for aerial vehicles," 3GPP TR 36.777 V15.0.0, Dec. 2017.

[27] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, "Deep sets," in *Proc. NeurIPS*, Long Beach, CA, Dec. 2017, pp. 3391–3401.

[28] U. Challita, W. Saad, and C. Bettstetter, "Interference management for cellular-connected UAVs: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2125–2140, Apr. 2019.

[29] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, Jul. 2023.

[30] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, vol. 80, Stockholm, Sweden, Jul. 2018, pp. 1861–1870.

[31] A. M. Khan, I. U. Rehman, N. Saeed, D. Sobnath, F. Khan, and M. A. K. Khattak, "Context-aware autonomous drone navigation using large language models (LLMs)," in *Proc. AAAI Symp. Series*, vol. 6, no. 1, 2025, pp. 102–107.

[32] M. Arana-Catania, A. Sonee, A.-M. Khan, K. Fatehi, Y. Tang, B. Jin, A. Soligo, D. Boyle, R. Calinescu, P. Yadav, H. Ahmadi, A. Tsourdos, W. Guo, and A. Russo, "Explainable reinforcement and causal learning for improving trust to 6G stakeholders," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 4101–4125, 2025.